

Algorithmic ‘Feature’ Pitch

Design and prototype an **algorithm-powered feature** that solves a problem **situated in an existing product, service, or domain**.

You are expected to think like a product and engineering team, identify a practical problem, and compare and contrast multiple valid algorithmic approaches to address it.

Project Goals

- Identify a **real-world problem** or suggest a feature to include in some already existing application or service.
 - Design and implement **at least two algorithmic solutions** using distinct paradigms studied in class.
 - Evaluate each approach’s **suitability within the context** of the problem
 - Demonstrate the feature in action.
 - Communicate the strengths, weaknesses, and trade-offs involved in the different algorithmic solutions.
-

Project Requirements

1. Problem Selection

- The problem must stem from a **realistic scenario** and be tied to a feature that could exist within an **actual application or system** (e.g., navigation, education, logistics, entertainment, productivity, etc.)
- You need **select and specify** your problem, and note that:
 - **Duplicate or highly similar projects** will be **penalized**
 - It is **your responsibility** to ensure your idea is unique and sufficiently distinct
 - You **may seek informal approval or feedback** from the instructor or TAs

2. Algorithmic Approaches

- You must implement **at least two distinct algorithmic paradigms** (e.g., brute force, greedy, dynamic programming, D&C, T&C, backtracking, ...etc.)
 - Each approach must be:
 - **Well-justified** based on the problem context
 - **Evaluated for complexity and efficiency**
 - **Analyzed for situational strengths and weaknesses**
 - Trivial or superficial use of brute-force **without context-specific justification** will be penalized
-

Deliverables

- **Brief Report (Max 4 pages)** must include:
 - **Problem Statement & Application Context**
 - Describe the real-world problem; Identify the feature and the application/system it fits into
 - **Formal Problem Definition:** Input, output, and constraints
 - **No less than two Algorithmic Approaches**
 - Explanation of design, and justification for choice
 - Time and space complexity with **contextual** pros and cons
 - **Situational Evaluation:** When and why one approach may outperform another or be more suitable in certain contexts
 - **Sample Test Cases** demonstrating varying situations.
 - **Include tables and graphs** showing how performance varies with input size (e.g., runtime vs input size), and how it aligns with the calculated complexity of each of the algorithms.
 - **Reflection:** Challenges, assumptions, limitations, alternative scenarios or solutions...etc.

- **Demonstration and Discussion**

- Demonstrate the prototype with sample input/output
- Clearly explain: The algorithms used, their situational benefits and trade-offs, the performance differences and real-world implications
- Discussions will be conducted in person during the last week of classes.
Stay alert for the appointment reservation announcement.

Evaluation Criteria

Component	Weight
Problem Statement & Application Validity	20%
Two Algorithmic Designs & Their Justification	25%
Situational Evaluation & Trade-off Analysis	20%
Code Functionality & Demo	20%
Clarity of Report & Discussion	15%

Note:

- You may reach out to the **instructor or TAs** for feedback on your problem idea or algorithm choices
 - However, **ensuring creativity, originality and depth is your responsibility**
 - Projects that are vague, superficial, or unoriginal will receive **reduced credit**
 - **Duplicate or highly similar projects** will receive **reduced credit**
-